

PROBAR

SESIÓN 08 · UNIDAD II · ETHEREUM Y CONTRATOS

DEFENDER SU CONTRATO Y PROBARLO

Primero lo defienden delante del curso. Después ven por qué una suite verde puede no estar probando nada.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Cinco minutos de apertura, cuatro bloques y una pausa de 10 minutos entre B y C. La clase empieza con los equipos al frente: 5 + 45 + 35 + 10 + 55 + 25 + 5 = 180 minutos.

A	**DEFENSAS** Cada equipo defiende su contrato proyectando el código. Siete minutos por equipo.	~45 min
B	**DEL NAVEGADOR AL TALLER** Por qué se sale de Remix, la forma de un proyecto Hardhat y cómo se escribe una prueba.	~35 min
C	**CAZAR MUTANTES** Qué mide la cobertura, qué no mide, y tres mutantes que hay que matar.	~55 min
D	**DESPLEGAR SIN GASTAR UN PESO** Ignition contra la red local, en vivo. Y qué cambia al apuntar a Sepolia.	~25 min

AL SALIR DE AQUÍ

Defender el contrato propio con el código en pantalla, y probarlo hasta que una suite verde signifique algo.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

01 ¿Qué promete su contrato, quién manda adentro y por dónde entra y sale el dinero?

02 ¿Qué hace que una prueba sirva, y no solo que esté en verde?

03 ¿Por qué una suite con 88 % de cobertura puede no detectar ni un error?

04 ¿Qué hace falta para desplegar y verificar, y qué parte de eso no cuesta un peso?

RA3 · patrones y estándares · RA4 · suite de pruebas automatizadas y despliegue con un entorno profesional

SIGLAS Y TÉRMINOS DE HOY

HARDHAT — *entorno de desarrollo*

Compila, prueba, despliega y verifica desde la terminal. Trae su propia red local, gratis e instantánea.

LOADFIXTURE — *estado de partida*

Despliega una vez, guarda una instantánea de la cadena y la restaura antes de cada prueba.

COBERTURA — *line coverage*

Porcentaje de líneas del contrato que alguna prueba ejecutó. No dice si alguien comprobó el resultado.

MUTANTE — *mutation testing*

Copia del contrato con un error puesto a propósito. Una suite que sirve tiene que fallar contra él.

BORDE — *edge case*

El valor exacto del límite: el último segundo, el peso justo, la segunda llamada seguida.

IGNITION — *despliegue declarativo*

Describe QUÉ se despliega, anota cada paso en un diario y no repite lo que ya hizo.

KEYSTORE — *almacén cifrado*

Donde Hardhat 3 guarda claves y URLs, con contraseña y fuera del repositorio.

VERIFICAR — *source code verification*

Subir la fuente al explorador para que recompile y compruebe que da el mismo bytecode.

TRES EVIDENCIAS Y UNA FRASE

Una suite con 88 % de cobertura puede no servir para nada. Hoy lo van a ver con números.

LO QUE SALE DE ESTA SESIÓN

01	LA DEFENSA	Cada equipo, al frente, con su contrato en pantalla. Se califica hoy mismo, con la rúbrica de la lámina A.3.
02	LA SUITE Y LOS MUTANTES	Diez pruebas o más para Recaudo, la cobertura medida, y los tres mutantes muertos con su explicación.
03	EL DESPLIEGUE LOCAL	La salida de Ignition con la dirección del contrato desplegado en la red local, hecho en vivo.

A

APROXIMADAMENTE 45 MINUTOS · 7 POR EQUIPO

DEFENSAS

Siete minutos por equipo, con el código en pantalla. Las cuatro pasadas de la Sesión 7, aplicadas al contrato propio, y una prueba que tiene que fallar.

QUÉ PASA EN CADA MINUTO

MINUTO	QUÉ HACE EL EQUIPO	QUÉ TIENE QUE VERSE EN LA PANTALLA
0:00 – 0:45	Pasada 1: qué promete el contrato. Solo los nombres públicos.	La lista de funciones y eventos. Ningún cuerpo abierto todavía.
0:45 – 1:45	Pasada 2: quién manda, y el día en que esa llave se pierde.	La línea del modificador o del require que da ese poder.
1:45 – 2:45	Pasada 3: por dónde entra el dinero y por dónde sale.	Cada payable y cada envío, señalados con el cursor.
2:45 – 4:00	La prueba propia que falla a propósito. Se corre ahí mismo.	La terminal, con la prueba en rojo y el mensaje del fallo.
4:00 – 6:00	Dos preguntas del docente, de las de la lámina A.4.	El archivo abierto, para buscar la respuesta en el código.
6:00 – 7:00	Cambio: el siguiente equipo conecta mientras se anota la nota.	Nada: es el minuto del cable y del proyector.

LAS DOS REGLAS DE LA DINÁMICA

El orden es por sorteo y se anuncia completo antes de empezar. A los 7 minutos se corta, sin excepción: elegir qué mostrar en cuatro minutos es parte de lo que se evalúa.

LAS CUATRO PASADAS, SOBRE SU CONTRATO

01	PASADA 1 · QUÉ PROMETE	Leer la lista de nombres públicos —funciones y eventos— y decir qué promete cada uno. Sin abrir ningún cuerpo: lo que se deduce de un nombre es una hipótesis, y se dice como hipótesis.
02	PASADA 2 · QUIÉN MANDA	Qué direcciones tienen poder, qué puede hacer cada una, y qué es exactamente lo que NO puede hacer. Y el día malo completo: si esa llave se pierde o se filtra, ¿qué queda atrapado?
03	PASADA 3 · EL DINERO	Por dónde entra (cada payable), dónde se queda (qué variable lo contabiliza) y por dónde sale. Quién dispara cada salida y con qué límite.
04	PASADA 4 · UNA PRUEBA QUE FALLE	Una prueba escrita por el equipo que falle A PROPÓSITO, corrida en pantalla. Tienen que decir qué regla queda sin cumplir cuando falla, y por qué eligieron esa.

[-] NO HAY NADA NUEVO QUE PREPARAR

Las pasadas 1 a 3 son las páginas 1 a 4 de la ficha de lectura que llenaron en la Sesión 7. La cuarta es la novedad de hoy.

LA RÚBRICA DE LA DEFENSA

CRITERIO	PESO	UN 5,0 SE OYE ASÍ	BAJA A 3,0 O MENOS SI…
¿Qué promete?	15 %	Lee la lista de nombres y dice qué promete cada uno, marcando lo que es hipótesis.	Lee el archivo de corrido, o explica cómo funciona cuando se le pidió qué promete.
¿Quién manda?	20 %	Nombra el rol, el poder exacto que tiene, y cuenta el día en que esa llave se pierde.	Responde «está protegido con OpenZeppelin», o no sabe qué queda atrapado ese día.
¿Y el dinero?	25 %	Señala con el cursor cada entrada y cada salida, y dice quién la dispara y con qué límite.	Describe el flujo de memoria, sin señalar ninguna línea del archivo.
La prueba que falla	40 %	Corre la prueba, falla, y explican qué regla quedó sin cumplir y por qué eligieron esa.	La prueba falla por otra razón —sintaxis, saldo, cuenta equivocada— y no lo notan. Sin prueba: 0.

La cuarta vale 40 %: es la única que no se puede improvisar en el pasillo, y por eso es la que de verdad separa a un equipo que conoce su contrato de uno que lo tiene.

«¿QUÉ PASA SI...?»

01	BORRAR	Señalen la línea que hace cumplir su regla principal. ¿Qué deja de ser cierto si yo la borro?
02	LA LLAVE	¿Quién se queda con el dinero o con el control si esa llave se pierde o se filtra hoy?
03	DOS VECES	¿Qué pasa si la misma cuenta llama dos veces seguidas a esa función?
04	EL RECEPTOR	¿Qué pasa si quien recibe el dinero es un contrato que rechaza el pago?
05	EL SEGUNDO EXACTO	¿Qué pasa un segundo antes, en el segundo exacto y un segundo después de ese plazo?
06	OTRA CUENTA	¿Qué pasa si llama una cuenta que no es la del dueño? ¿Y si quien llama es un contrato?

Y la última, siempre: digan una cosa que su contrato NO puede impedir, y por qué siguen pensando que vale la pena.

LO QUE SE HACE, Y LO QUE NO

01	NO SE REGAÑA	Nadie pierde la defensa por esto. Se sigue con el resto de la rúbrica y se califica completo lo que sí trajeron.
02	SE DICE EN VOZ ALTA	«¿Qué regla querrían comprobar, y en qué línea de su contrato vive?» Treinta segundos, sin código y sin terminal.
03	QUEDA ESCRITO	El docente anota la regla que el equipo dijo. Esa es la prueba que debe entregar, no otra cualquiera.
04	CON FECHA	Se entrega antes de la Sesión 9, por el canal del curso: el archivo de prueba y la captura de la corrida en rojo.

POR QUÉ DECIRLA EN VOZ ALTA YA VALE ALGO

Un equipo que sabe qué quiere comprobar está a una tarde de la prueba. El que no lo sabe, no lo está, y conviene saberlo hoy.

LO QUE LAS DEFENSAS DEBIERON DEJAR

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 De todos los contratos que vieron hoy, ¿cuál tenía la regla principal más clara, y en qué línea vivía?

02 ¿Qué equipo contó el día malo completo de su llave, y qué quedaba atrapado ese día?

03 ¿Una prueba que falla porque la cuenta no tiene saldo demuestra algo sobre la regla del contrato?

04 Sobre su propio contrato: ¿qué pregunta de la lámina A.4 les costó más, y por qué esa?

B

APROXIMADAMENTE 35 MINUTOS

DEL NAVEGADOR AL TALLER

Remix sirve para explorar. No sirve para versionar, ni para correr cien pruebas en lote, ni para desplegar dos veces igual. Hoy se cambia de herramienta.

LO QUE REMIX NO PUEDE HACER BIEN

Remix sigue siendo excelente para explorar un contrato y para enseñar. Pero hay seis cosas que un equipo necesita y que ahí no se pueden hacer.

LO QUE HACE FALTA	EN REMIX	EN HARDHAT
Guardar la historia del código	Archivos dentro del navegador.	Git: ramas, revisión, quién cambió qué.
Correr cien pruebas en lote	A mano, clic por clic.	Un comando, y segundos.
Desplegar dos veces igual	Cada vez a mano, y distinto.	Un módulo con parámetros, repetible.
Saber qué líneas nadie ejecutó	No existe.	Informe de cobertura, por número de línea.
Probar qué pasa en una semana	Imposible: el tiempo es el real.	Se adelanta el reloj de la red local.
Guardar claves con cuidado	Depende de la billetera del navegador.	Almacén cifrado, fuera del proyecto.

Ningún equipo serio despliega desde el navegador. No porque Remix sea malo, sino porque un despliegue tiene que poder repetirse exactamente igual, y eso un clic no lo garantiza.

ANATOMÍA DE UN PROYECTO HARDHAT

```
laboratorios-evm/
├── contracts/          los contratos .sol
├── test/               las pruebas .test.js
├── ignition/
│   ├── modules/        CÓMO se despliega
│   └── parametros/     CON QUÉ valores, por red
├── scripts/            utilidades sueltas
├── hardhat.config.js   compilador y redes
├── package.json        versiones EXACTAS
└── .gitignore          lo que NUNCA se sube
```

HARDHAT.CONFIG.JS · EL DEL CURSO, RECORTADO

```
export default {
  plugins: [toolbox],
  solidity: {
    version: "0.8.34",
    settings: {
      optimizer: { enabled: true, runs: 200 },
    },
  },
  networks: {
    sepolia: {
      type: "http",
      url: configVariable("SEPOLIA_RPC_URL"),
      accounts: [configVariable("SEPOLIA_PRIVATE_KEY")],
    },
  },
};
```

configVariable NO contiene el secreto: le dice a Hardhat que lo busque en el almacén cifrado, y solo cuando se use esa red. Las pruebas locales nunca lo piden, así que se puede trabajar todo el semestre sin configurar ninguna clave.

UNA CADENA COMPLETA EN SU COMPUTADOR

LO QUE TRAE	QUÉ PERMITE HACER
20 cuentas con 10 000 ETH	Simular a Ana, a Beto, al dueño y a un atacante sin pedirle fondos a nadie.
Minado instantáneo	Cada transacción entra en un bloque al momento: las pruebas corren en milisegundos.
Control del reloj	Adelantar siete días para probar un plazo, sin esperar una semana.
Instantáneas del estado	Guardar la cadena y volver a ese punto: es la base de las fixtures.
Errores completos	Dice qué error personalizado revirtió y con qué argumentos, no «transaction failed».
npx hardhat node	La misma red como servidor en localhost:8545, para conectar una billetera o una interfaz.

⚠ **LAS CUENTAS DE LA RED LOCAL SON PÚBLICAS**

Sus claves privadas están impresas en la documentación. Nunca se usan en una red real.

LA FORMA DE UN ARCHIVO DE PRUEBAS

TEST/S07/SUBASTA.TEST.JS · CÓDIGO REAL DEL REPOSITORIO, RECORTADO

```
import { expect } from "chai";
import { network } from "hardhat";
const { ethers, networkHelpers } = await network.create();   // Hardhat 3: create()
const { time, loadFixture } = networkHelpers;
const HORA = 60 * 60;
const MINIMA = ethers.parseEther("1");

async function desplegar() {                          // la fixture LLEVA NOMBRE
  const [dueno, ana, beto, carla] = await ethers.getSigners();
  const s = await ethers.deployContract("Subasta", [HORA, MINIMA]);
  return { s, dueno, ana, beto, carla };
}

describe("S07 · Subasta · pujar", () => {                          // el grupo
  it("acepta la primera puja desde el mínimo", async () => {          // el caso
    const { s, ana } = await loadFixture(desplegar);
    await expect(s.connect(ana).pujar({ value: MINIMA }))
      .to.emit(s, "NuevaPuja").withArgs(ana.address, MINIMA);     // qué se exige
  });
});
```

Se lee como una frase: «Subasta, al pujar, acepta la primera puja desde el mínimo». Si no se lee así, está mal nombrado.

CADA PRUEBA EMPIEZA DE CERO, SIN PAGARLO

01	PRIMERA LLAMADA	loadFixture ejecuta desplegar(): despliega el contrato y guarda una instantánea de toda la cadena.
02	LLAMADAS SIGUIENTES	No vuelve a desplegar: restaura la instantánea. Milisegundos en lugar de segundos.
03	AISLAMIENTO	Lo que una prueba cambió —aportes, saldos, el reloj adelantado— no existe para la siguiente.

⚠ EL ERROR CLÁSICO, Y POR QUÉ ES TAN DIFÍCIL DE ENCONTRAR

Desplegar una sola vez en un before() compartido y dejar que las pruebas se afecten entre sí. Una prueba adelanta el reloj una semana; la siguiente encuentra el plazo vencido y falla sin razón aparente. La suite pasa en un orden y falla en otro, y se pierden horas buscando el error en el contrato, que está bien. Con loadFixture ese problema no puede ocurrir.

QUIÉN FIRMA, Y EN QUÉ MOMENTO

```
const [dueno, ana, beto, carla] = await ethers.getSigners();   // las 20 cuentas

await s.connect(ana).pujar({ value: eth(1) });          // firma Ana, no el dueño
await s.connect(beto).pujar({ value: eth(2) });         // firma Beto

await time.increase(HORA);                              // pasa una hora
await time.increaseTo(fin);                             // mina un bloque EN fin
await time.setNextBlockTimestamp(fin);                  // el PRÓXIMO bloque, en fin

await expect(s.connect(ana).pausar())                   // una cuenta sin permiso
  .to.be.revertedWithCustomError(s, "OwnableUnauthorizedAccount");
```

CONNECT(CUENTA)

Cambia quién firma. Sin él, todo lo hace el dueño y las pruebas de permisos pasan por accidente.

TIME

Mueve el reloj de la red local, nunca el de una red real. Viene de networkHelpers.

REVERTEDWITHCUSTOMERROR Y WITHARGS

```
// test/s07/Subasta.test.js · la reversión, con su error y sus argumentos
it("rechaza una primera puja por debajo del mínimo", async () => {
  const { s, ana } = await loadFixture(desplegar);
  await expect(s.connect(ana).pujar({ value: eth("0.5") }))
    .to.be.revertedWithCustomError(s, "PujaInsuficiente").withArgs(MINIMA, eth("0.5"));
});

it("no acepta pujas después del cierre", async () => {
  const { s, ana } = await loadFixture(desplegar);
  await time.increase(HORA);
  await expect(s.connect(ana).pujar({ value: eth(5) }))
    .to.be.revertedWithCustomError(s, "SubastaCerrada");
});
```

⚠ **POR QUÉ NO VALE DECIR SOLO «REVIRTIÓ»**

No usen .to.be.reverted a secas: está desaconsejado en Hardhat 3 y pasa aunque el contrato revierta por la razón equivocada: falta de saldo, cuenta sin permiso, otra regla que se disparó primero. revertedWithCustomError exige el error exacto, y withArgs exige además los valores con los que se revirtió. Esa es la diferencia entre una prueba y una ilusión de prueba.

WITHARGS, Y EL DINERO QUE SE MOVIÓ

```
// el evento, con los argumentos exactos
await expect(s.connect(carla).finalizar())
  .to.emit(s, "SubastaFinalizada").withArgs(ana.address, eth(3));

// el saldo de una cuenta, y de varias a la vez (ethers va PRIMERO)
await expect(s.connect(ana).retirar())
  .to.changeEtherBalances(ethers, [ana, s], [eth(1), -eth(1)]);

// el perdedor NO recibe su dinero al ser superado: su saldo cambia en 0
await expect(s.connect(beto).pujar({ value: eth(2) }))
  .to.changeEtherBalance(ethers, ana, 0n);

// y lo que quedó guardado se lee como cualquier variable pública
expect(await s.pendientes(ana.address)).to.equal(eth(1));
```

⚠ **DOS TRAMPAS COMPROBADAS EN EL REPOSITORIO**

changeEtherBalance ya descuenta el gas, así que se compara el monto exacto y no hay que restarle nada. Y no se encadena con .to.emit sobre la misma transacción: se envía dos veces y la segunda falla.

LAS CINCO FAMILIAS DE PRUEBAS

FAMILIA	QUÉ PREGUNTA RESPONDE	EJEMPLO REAL DE TEST/S07/SUBASTA.TEST.JS
Caso feliz	¿Hace lo que promete cuando todo va bien?	Una puja válida cambia mejorPostor y emite NuevaPuja.
Reversión	¿Se niega a hacer lo que no debe?	Pujar por debajo del mínimo revierte con PujaInsuficiente.
Permisos	¿Solo quien debe puede hacerlo?	Solo el dueño pausa: el resto recibe OwnableUnauthorizedAccount.
Eventos	¿Le cuenta al mundo lo que pasó, con los datos correctos?	SubastaFinalizada, con el ganador y el monto exactos.
Bordes	¿Qué pasa exactamente en el límite?	La puja que supera en exactamente el 1 %, ni un wei menos.

Las cuatro primeras dan cobertura. La quinta es la que atrapa los errores.

ANTES DE LA PAUSA: CUATRO PREGUNTAS

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 ¿Por qué una fixture tiene que ser una función con nombre, y qué pasa si se le pasa una flecha anónima?

02 ¿Qué comprueba de verdad una prueba que dice solo .to.be.reverted, y qué no?

03 Si quieren probar que una cuenta sin permiso no puede pausar, ¿qué dos cosas tiene que tener esa prueba?

04 ¿Qué comando corre las pruebas, y cuál de los dos lenguajes está en test/ y cuál en contracts/?

PAUSA · 10 MINUTOS

DIEZ MINUTOS

Al volver: una suite en verde, 88 % de cobertura, y tres errores que nadie ve.

Mientras tanto, dos cosas: abran una terminal en laboratorios-evm y corran npx hardhat --version —tiene que decir 3.16.0—, y tengan a mano el contrato del laboratorio, contracts/s08/Recaudo.sol. El bloque C empieza escribiendo pruebas, no instalando cosas.

C

APROXIMADAMENTE 55 MINUTOS · EN PAREJAS

CAZAR MUTANTES

Trece pruebas, 88 % de cobertura y tres errores vivos. Tres pruebas más y los tres mueren. El bloque empieza con los números y termina con ustedes escribiendo esas pruebas.

RECAUDO: UNA VACA CON META Y PLAZO

Un semillero recoge aportes para un proyecto. Hay una meta y una fecha de cierre. Lo que pase después depende de una sola cosa: si se alcanzó la meta o no.

01	MIENTRAS HAY PLAZO	Cualquiera aporta con aportar(). El contrato anota cuánto puso cada quien y el total acumulado.
02	SE VENCE EL PLAZO	Se acabaron los aportes. El contrato pasa a Exitoso si el total llegó a la meta, o a Fallido si no llegó.
03	SI FUE EXITOSO	El beneficiario —el dueño del contrato— reclama todo con reclamarFondos(). Una sola vez.
04	SI FUE FALLIDO	Cada aportante recupera exactamente lo suyo con reembolsar(). Ni más, ni el de otro.

POR QUÉ ESTE CASO Y NO OTRO

Tiene dinero, plazo y dos finales opuestos. Casi todos los errores de contratos viven en esos tres sitios.

TRES FUNCIONES Y NUEVE FORMAS DE DECIR QUE NO

```
function aportar() external payable {
    if (block.timestamp >= fin) revert PlazoVencido();
    if (msg.value == 0)         revert AporteCero();
    aportes[msg.sender] += msg.value;
    totalRecaudado      += msg.value;
    emit Aporte(msg.sender, msg.value, totalRecaudado);
}

function reclamarFondos() external onlyOwner {
    if (block.timestamp < fin)    revert PlazoVigente();
    if (totalRecaudado < meta)    revert MetaNoAlcanzada();
    if (fondosReclamados)         revert YaReclamado();
    fondosReclamados = true;                  // efectos antes del envío
    (bool ok, ) = payable(owner()).call{value: address(this).balance}("");
}

function reembolsar() external {
    if (block.timestamp < fin)    revert PlazoVigente();
    if (totalRecaudado >= meta)   revert MetaAlcanzada();
    uint256 monto = aportes[msg.sender];
    if (monto == 0)               revert NadaQueReembolsar();
    aportes[msg.sender] = 0;                  // en cero ANTES de enviar
    (bool ok, ) = payable(msg.sender).call{value: monto}("");
}
```

Cuenten las comparaciones: siete. Cada una es una regla, y cada regla tiene un borde. Ahí se esconden los tres errores.

QUÉ MIDE, Y QUÉ NO MIDE

```
npx hardhat test test/s08/Recaudo.test.js --coverage
```

QUÉ MIDE

- Qué líneas del contrato ejecutó alguna prueba.
- Qué líneas no se ejecutaron nunca: las lista por número.
- Sirve para encontrar funciones o reversiones que nadie probó.

QUÉ NO MIDE

- Si la prueba comprobó el resultado: una prueba sin expect da cobertura.
- Si se probó el valor exacto del límite, o solo uno cómodo.
- Si el contrato hace lo que el negocio necesita.

La cobertura dice dónde NO hay pruebas. No dice que las que hay sirvan.

UNA FILA, TRES COLUMNAS Y UNA LISTA

SALIDA REAL CON LA PLANTILLA DEL LABORATORIO, RECORTADA

File Path	Line %	Statement %	Uncovered Lines
contracts\s08\Recaudo.sol	11.54	15.79	49, 55-56, 60, 63-64, 66, 70-72, 74-75, 77-78, 80,…
contracts\s07\Subasta.sol	18.18	0.00	59-61, 67-69, 74-75, 83, 89-91, 93-96, 98-99, 101,…
Total	18.49	0.73	

01 **BUSQUEN SU FILA**

El informe lista TODOS los contratos del repositorio. La única que importa es la de contracts\s08\Recaudo.sol.

02 **IGNOREN EL TOTAL**

18,49 % de Total no significa nada: suma contratos de otras sesiones que esta suite ni toca.

03 **LEAN LAS LÍNEAS**

Cada número es una pregunta. Abran Recaudo.sol en esa línea y escriban la prueba que la ejecutaría.

Son cifras reales: es lo que da la plantilla de dos pruebas del laboratorio. Queda además un informe navegable en coverage/html.

¿MIS PRUEBAS NOTARÍAN UN ERROR?

La idea es simple y un poco desagradable: se toma el contrato, se le mete un error pequeño a propósito —eso es un mutante— y se corre la misma suite, sin tocar ni una prueba. Lo que pase después dice si la suite sirve.

MUTANTE MUERTO

Al menos una prueba falla contra él. La suite vio el error: ese error no habría llegado a producción.

MUTANTE VIVO

Todas las pruebas siguen en verde. El error está ahí y nadie lo nota: así se despliega un contrato roto.

POR QUÉ ESTO IMPORTA MÁS AQUÍ QUE EN UNA APLICACIÓN WEB

La cobertura dice dónde no hay pruebas; la mutación dice si las que hay sirven. Son preguntas distintas y la segunda es la que importa, porque un contrato desplegado no se puede parchear. Lo que esta tarde es un mutante de juguete, en producción es una línea mal escrita que custodia dinero para siempre, y la suite en verde es lo único que había entre ella y el despliegue.

QUÉ SON EXACTAMENTE ESOS MUTANTES

En la carpeta del laboratorio, al lado de Recaudo.sol, hay tres archivos más: RecaudoMutante1, 2 y 3. Son copias del mismo contrato, idénticas línea por línea salvo UNA.

LO QUE HAY QUE SABER	EL DETALLE
Los escribió el docente	No son un error que apareció solo: cada uno tiene un error metido a propósito, y los tres son distintos entre sí.
Uno solo cada uno	Un cambio mínimo: una comparación, un número, una línea que falta. El contrato compila igual y funciona igual casi siempre.
No se dice cuál ni dónde	Ni en el deck, ni en la guía, ni en el encabezado del archivo. Averiguarlo es el ejercicio.
Están abiertos	Pueden abrirlos: nadie se lo impide. Pero el que los abre se queda sin laboratorio, y en el parcial no va a haber un archivo que abrir.
Se corren solos	El cazador toma SU suite —sin tocar ni una prueba— y la corre contra los cuatro contratos, uno tras otro.

⚠ **Y POR ESO LA REGLA DEL LABORATORIO ES RARA PERO JUSTA**

No se cambia ni una línea de ningún contrato. Lo único que se escribe son pruebas.

DOS MUTANTES, MEDIDOS

MUTANTE SOBRE SUBASTA.SOL	CONTRA LAS 23 PRUEBAS DE LA S7	POR QUÉ
`INCREMENTO_MINIMO_BPS = 10 en vez de 100`	MUERTO · 21 pasan, 2 fallan	Dos pruebas comparan el valor exacto: 2,02 ETH y 0,02 ETH. El 10 % no da eso.
`Comentar el emit Retiro(...) de retirar()`	VIVO · 23 pasan, 0 fallan	Ninguna de las 23 comprueba ese evento. La línea se ejecuta; nadie mira qué emite.

LO QUE ENSEÑA EL MUTANTE QUE SOBREVIVIÓ

Subasta.sol tiene 97,06 % de cobertura de líneas y aun así, sin ese emit, una interfaz que escuche el evento Retiro deja de enterarse de los retiros y ninguna prueba lo nota. La línea estaba cubierta; lo que nadie comprobaba era lo que emitía. Arreglarlo cuesta una línea, la de B.8: .to.emit(s, "Retiro").withArgs(ana.address, eth(1)).

Las dos corridas se hicieron esta semana en una copia aislada del repositorio, contra test/s07/Subasta.test.js. Ejercicio relámpago, treinta segundos: ¿qué prueba agregarían para matar al segundo?

88 % DE COBERTURA Y TRES ERRORES VIVOS

Preparando el laboratorio se escribió una suite de referencia para Recaudo y tres copias del contrato, cada una con un error distinto. Después se quitaron SOLO las tres pruebas de borde. Esto es lo que salió.

	SIN LAS PRUEBAS DE BORDE	SUITE COMPLETA	LA DIFERENCIA
Pruebas	13	16	+3 pruebas de borde
Cobertura de líneas	88,46 %	92,31 %	+3,85 puntos
Sentencias (Statement %)	92,11 %	94,74 %	+2,63 puntos
Líneas sin cubrir	60, 78, 93	78, 93	una línea menos
Los tres mutantes	VIVOS · 0 fallan	MUERTOS · 1 falla cada uno	de no ver nada a verlo todo

Tres pruebas separan «no detecta nada» de «detecta todo».

DÓNDE VIVEN LOS ERRORES

BORDE	LA PREGUNTA QUE HAY QUE HACERLE AL CONTRATO	POR QUÉ AHÍ
Tiempo	¿Qué pasa un segundo antes, en el segundo exacto y un segundo después de cada plazo?	Una comparación mal escrita deja entrar o salir justo en el límite, y ninguna prueba «normal» llega a ese instante.
Umbral	¿Qué pasa con el valor exacto de cada límite: ni un wei más, ni uno menos?	Los casos cómodos siempre quedan lejos del límite, así que nadie los prueba y ahí nadie mira.
Repetición	¿Qué pasa si la misma cuenta llama dos veces seguidas a la misma función?	Un efecto que debería ocurrir una sola vez puede repetirse si el estado no cambió antes.
Cero y vacío	¿Qué pasa con 0, con la dirección cero, con un texto vacío?	Son valores perfectamente válidos para la EVM y absurdos para el negocio.
Quién llama	¿Qué pasa si llama otra cuenta, o si quien llama es un contrato?	Basta una función a la que se le olvidó el modificador.

Para cada función, recorrer las cinco filas y escribir la prueba que falta. Así se encuentra lo que la cobertura no ve.

¿EN QUÉ SEGUNDO CAE MI TRANSACCIÓN?

La Subasta de la S7 dice: if (block.timestamp >= fin) revert SubastaCerrada(). ¿Qué pasa si alguien puja en el segundo exacto de fin? Para probarlo hay que poder llevar una transacción a ese segundo, y no a «una hora después».

HERRAMIENTA	QUÉ HACE CON EL RELOJ	LA PUJA SIGUIENTE CAE EN	SIRVE PARA PROBAR
`time.increaseTo(fin)`	Mina un bloque vacío con marca fin.	fin + 1	«después del cierre»
`time.setNextBlockTimestamp(fin)`	No mina nada: fija la marca del PRÓXIMO bloque.	fin, exacto	«en el segundo exacto»
`setNextBlockTimestamp(fin - 1n)`	Lo mismo, un segundo antes.	fin - 1	«un segundo antes»

[-] LA REGLA, PARA CUALQUIER PLAZO DE CUALQUIER CONTRATO

Por cada comparación con el tiempo, tres pruebas: un segundo antes, el segundo exacto y un segundo después. Una prueba con «una semana después» nunca llega al límite: por eso casi nadie lo prueba, y por eso ahí viven los errores.

EL CONTRATO, LA PLANTILLA Y LA REGLA

```
npx hardhat --version                                  # 3.16.0
mkdir test\s08                                         # Git Bash: mkdir -p test/s08
copy andamiaje\s08\Recaudo.test.js test\s08\           # Git Bash: cp ...
npx hardhat test test/s08/Recaudo.test.js              # 2 passing: los dos ejemplos
```

FINAL DEL RECAUDO	CONDICIÓN, UNA VEZ VENCIDO EL PLAZO	QUÉ SE PUEDE HACER
Exitoso	totalRecaudado >= meta	El dueño reclama los fondos, una sola vez.
Fallido	totalRecaudado < meta	Cada aportante recupera exactamente lo suyo.

01 **LEER EL CONTRATO** — contracts/s08/Recaudo.sol: tres funciones y dos finales. Subrayen cada <, >= y ==: esa es su lista de bordes.

02 **NO ABRIR LOS MUTANTES** — Hasta que los tres estén muertos. Abrirlos antes cambia el ejercicio por copiar una diferencia.

ESCRIBIR LA SUITE: DIEZ PRUEBAS O MÁS

GRUPO	MÍNIMO	QUÉ NO OLVIDAR
despliegue	2	Parámetros inválidos en el constructor.
aportar	3	El evento con argumentos exactos. Los dos extremos del plazo.
recaudo exitoso	3	Permisos. Doble reclamo. El saldo exacto que recibe el dueño. El límite de la meta.
recaudo fallido	3	Cada quien lo suyo. Quien no aportó. El dueño no se lleva nada. Las repeticiones.

[-] UNA PRUEBA FALLA POR LA RAZÓN CORRECTA, O NO PRUEBA NADA

Antes de dar por buena una prueba de reversión, pregúntense qué OTRA cosa podría hacerla revertir. Si la segunda llamada revierte porque el contrato se quedó sin dinero y no por la regla que querían comprobar, la prueba pasa por accidente y seguiría pasando si la regla no existiera. Diseñen el escenario para que SOLO la regla pueda detener la transacción, y exijan el error exacto con revertedWithCustomError.

MEDIR LA COBERTURA Y LEER LO QUE FALTA

```
npx hardhat test test/s08/Recaudo.test.js --coverage

‖ contracts\s08\Recaudo.sol   | 11.54   | 15.79   | …   ← esto es lo que da la plantilla
```

01	SU FILA	contracts\s08\Recaudo.sol, columna Line %. La meta del curso es 80 o más.
02	CADA NÚMERO	Abran Recaudo.sol en esa línea y pregúntense qué prueba la ejecutaría. Escríbanla.
03	LO QUE QUEDE	Si solo quedan líneas de revert EnvioFallido, está bien: exigen un receptor que rechace el pago. Anótenlo.

Cómo saben que esta parte terminó: diez pruebas o más en verde, y Line % de 80 o más en la fila de Recaudo.sol. Captura de pantalla, que es la evidencia. Y una advertencia que la parte 3 va a demostrar: llegar al 80 % es la parte fácil, y no basta.

CAZAR LOS TRES MUTANTES

SALIDA REAL CON LA PLANTILLA DE DOS PRUEBAS

```
node scripts/s08/cazar-mutantes.js

  Suite: test/s08/Recaudo.test.js

  ✔ Recaudo (correcto)      2 pruebas en verde

  ✗ VIVO    RecaudoMutante1   2 pasan · 0 fallan
  ✗ VIVO    RecaudoMutante2   2 pasan · 0 fallan
  ✗ VIVO    RecaudoMutante3   2 pasan · 0 fallan

  3 mutante(s) sobrevive(n). Hay un error que su suite no ve: piensen en los bordes.
```

01	**MIENTRAS HAYA VIVOS**	Volver a la tabla de bordes de C.6 y al subrayado de Recaudo.sol. Escribir la prueba que falta. No abrir el mutante.
02	**CUANDO MUERAN LOS TRES**	Ahora sí: abrir cada mutante, encontrar la diferencia y anotar qué prueba lo mató y por qué esa.

SI A LOS 35 MINUTOS NADIE MATÓ EL PRIMERO

Minuto 35 del bloque: si nadie mató al primero, se mata entre todos.

01	EN EL TABLERO	El docente escribe las comparaciones de Recaudo.sol y la sala propone el borde, en voz alta, sin computador.
02	SE ESCRIBE JUNTOS	La prueba se teclea en el proyector, se corre, y el primer mutante muere delante de todos.
03	LOS OTROS DOS	Quedan de trabajo autónomo, con la misma evidencia: qué prueba lo mató y por qué esa.

Nadie se va sin haber visto morir un mutante. Un laboratorio que no llega al final no enseña la lección, y la lección es lo que se evalúa, no la velocidad.

ERRORES FRECUENTES Y EVIDENCIA

SÍNTOMA	CAUSA, Y QUÉ HACER
El script dice «No se encontraron pruebas»	La suite no está en test/s08/Recaudo.test.js.
Una prueba de plazo pasa contra los tres mutantes	Nunca llega al segundo exacto: vuelvan a C.7.
Una reversión pasa, pero por otra razón	Rediseñen el escenario y exijan el error exacto.
loadFixture falla con una flecha anónima	La fixture necesita nombre: async function desplegar().
Cambié el contrato y el mutante murió	No se toca el contrato. Se escriben pruebas.

CRITERIO DE LA GUÍA	QUÉ SE ENTREGA	PESO
Suite	Diez pruebas o más de las cinco familias, en verde contra el contrato correcto.	25 %
Cobertura	Captura con Line % de 80 o más, y una explicación de cada línea sin cubrir.	15 %
Mutantes	Los tres muertos, con qué prueba mató a cada uno y por qué la cobertura no lo veía.	35 %
Despliegue	Hoy: la salida de Ignition en local. Queda en deuda hasta desplegar en Sepolia.	25 %

LO QUE EL LABORATORIO DEBIÓ DEJAR CLARO

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 Su suite pasó del 80 % de cobertura antes de matar a los tres mutantes. ¿Qué les dice eso sobre el requisito del 80 %?

02 Para una de sus pruebas de borde: ¿qué otra razón podría haberla hecho fallar, y cómo descartaron esa razón?

03 ¿Qué líneas de Recaudo.sol les quedaron sin cubrir, y por qué es aceptable (o no lo es)?

04 Si la cobertura sube 3,85 puntos y la detección pasa de 0 a 3 errores de 3, ¿para qué sirve exactamente la cobertura?

D

APROXIMADAMENTE 25 MINUTOS · EN VIVO

DESPLEGAR SIN GASTAR UN PESO

El despliegue en la red local se hace ahora, en vivo y gratis. El salto a Sepolia y la verificación del código se explican: hoy no hay ETH de prueba, y eso queda dicho como deuda.

IGNITION: QUÉ SE DESPLIEGA, Y CON QUÉ VALORES

IGNITION/MODULES/RECAUDO.JS · EL ARCHIVO REAL

```
import { buildModule } from "@nomicfoundation/hardhat-ignition/modules";

export default buildModule("RecaudoModulo", (m) => {
  const meta = m.getParameter("meta", 10n * 10n ** 18n);
  const duracion = m.getParameter("duracionSegundos", 7 * 24 * 60 * 60);

  const recaudo = m.contract("Recaudo", [meta, duracion]);
  return { recaudo };
});
```

PARAMETROS/RECAUDO-SEPOLIA.JSON

```
{
  "RecaudoModulo": {
    "meta": "5000000000000000",
    "duracionSegundos": 1800
  }
}
```

- Describe QUÉ se despliega, no los pasos: Ignition decide el orden y los lotes.
- El segundo argumento de getParameter es el valor por omisión: con eso funciona en la red local sin pasar ningún archivo.
- El mismo módulo sirve para la red local y para Sepolia. Lo único que cambia son los parámetros: 0,005 ETH y 30 minutos en lugar de 10 ETH y 7 días.
- Los números grandes van como texto en el JSON, porque $5 \cdot 10^{15}$ no cabe exacto en un número de JavaScript.

DESPLEGAR EN LA RED LOCAL

SALIDA REAL, MEDIDA AL PREPARAR LA SESIÓN · SIN LOS EMOJIS DE LA TERMINAL

```
npx hardhat node                        # en OTRA terminal: la red local como servidor
npx hardhat ignition deploy ignition/modules/Recaudo.js --network localhost

Hardhat Ignition

Deploying [ RecaudoModulo ]

Batch #1
  Executing RecaudoModulo#Recaudo...
  Executed RecaudoModulo#Recaudo

[ RecaudoModulo ] successfully deployed

Deployed Addresses

RecaudoModulo#Recaudo - 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Cómo se sabe que funcionó: dice «successfully deployed» y da una dirección. Esa dirección es siempre la misma en una red local recién arrancada —primera cuenta, nonce 0—, y eso es señal de que la red es determinista, no de que algo esté mal. Sin npx hardhat node, el mismo comando funciona igual pero contra una red en memoria que se borra al terminar.

QUÉ QUEDÓ REGISTRADO DEL DESPLIEGUE

LO QUE IGNITION ESCRIBIÓ EN EL DISCO

```
ignition/deployments/chain-31337/
├── deployed_addresses.json   la dirección desplegada
├── journal.jsonl             el diario de lo que se hizo
├── artifacts/                el ABI del contrato
└── build-info/               fuente y configuración del compilador

{ "RecaudoModulo#Recaudo": "0x5FbDB2315678afecb367f032d93F642f64180aa3" }
```

EL RECIBO, LEÍDO DEL NODO

```
desde      0xf39F…2266  (cuenta 1)
contrato   0x5FbDB…0aa3
bloque     1
estado     SUCCESS
gas        515 666
bytecode   2 014 bytes
args       10 ETH · 604 800 s
```

01 **LO QUE SE GUARDA**

La dirección, el diario, el ABI y la configuración exacta del compilador. Las cuatro cosas hacen falta para verificar.

02 **LO QUE SE SUBE A GIT**

El de Sepolia (chain-11155111) sí, como evidencia del Avance 1. El de la red local (chain-31337) lo excluye el .gitignore.

Las cifras del recibo se midieron al preparar esta lámina, pidiéndole al nodo local el recibo de esa transacción.

CORRER EL MISMO COMANDO DOS VECES

SEGUNDA CORRIDA · SALIDA REAL

```
npx hardhat ignition deploy ignition/modules/Recaudo.js --network localhost
[ RecaudoModulo ] Nothing new to deploy based on previous execution stored in .\ignition\deployments\chain-31337
```

UN SCRIPT SUELTO

Si se cae después de desplegar y antes de guardar la dirección, al repetirlo despliega otra vez. Quedan dos contratos, el dinero gastado dos veces, y nadie sabe cuál es el bueno.

IGNITION

Escribe cada paso en el diario antes de darlo. Al repetir el comando lee el diario, ve que ya está hecho y no vuelve a desplegar: dice «Nothing new to deploy».

Por eso el mismo comando se puede repetir sin miedo. Y si una verificación falla por tiempo de espera, se vuelve a lanzar tal cual: Ignition sabe que el contrato ya está desplegado y solo reintenta lo que falta.

QUÉ CAMBIA AL APUNTAR A SEPOLIA

LO QUE CAMBIA	EN LA RED LOCAL · LO DE HOY	EN SEPOLIA · LO PENDIENTE
La red	Nada, o --network localhost.	--network sepolia, y una URL de nodo con clave de API.
La clave	Las 20 cuentas de la red local, públicas.	La clave privada de la billetera del curso, en el keystore.
El gas	Gratis e instantáneo.	ETH de prueba de verdad, y una espera de segundos o minutos.
El resultado	Se borra al apagar la red.	Queda para siempre, y cualquiera lo puede leer.
La verificación	No aplica: no hay explorador.	--verify sube la fuente al explorador y publica el código.

LO QUE NO CAMBIA, Y POR ESO EL TRABAJO DE HOY SIRVE

El módulo es el mismo archivo. El contrato es el mismo. La suite es la misma. Lo único que se agrega son la red, la clave y la paciencia.

LA CLAVE PRIVADA NUNCA VIVE EN UN ARCHIVO

```
npx hardhat keystore set SEPOLIA_RPC_URL        # la primera vez pide crear una contraseña
npx hardhat keystore set SEPOLIA_PRIVATE_KEY    # la de la billetera DEL CURSO
npx hardhat keystore set ETHERSCAN_API_KEY
npx hardhat keystore list                       # muestra los NOMBRES, nunca los valores
```

QUÉ HACE EL ALMACÉN

- Guarda los valores cifrados, con una contraseña que se pide al usarlos.
- Vive en un archivo de la carpeta de usuario, fuera del proyecto y fuera de Git.
- configVariable los busca solo cuando una tarea los necesita de verdad.

SI UNA CLAVE SE SUBE A GITHUB

- Hay programas que recorren los repositorios públicos buscando claves.
- ConsenSys documenta billeteras vaciadas «en segundos».
- Borrar el archivo no basta: la clave queda en el historial de Git para siempre.

Si ocurre, la clave se da por perdida: se mueven los fondos a una billetera nueva y esa no se vuelve a usar nunca. No se «arregla» con un commit.

EN LA CADENA SOLO HAY BYTECODE

Lo que quedó desplegado son 2 014 bytes de números: ni un nombre de función, ni un comentario, ni el nombre del archivo. Verificar es darle al explorador lo necesario para que vuelva a compilar la fuente y compruebe que le sale exactamente ese bytecode.

QUÉ SE LE ENTREGA AL EXPLORADOR	DE DÓNDE SALE EN ESTE PROYECTO
El código fuente completo	contracts/s08/Recaudo.sol, y todo lo que importa, OpenZeppelin incluido.
La versión exacta del compilador	hardhat.config.js: 0.8.34, que es 0.8.34+commit.80d5c536.
La configuración del optimizador	hardhat.config.js: enabled true, runs 200.
Los argumentos del constructor	Los del módulo de Ignition: 10 ETH de meta y 604 800 segundos.

[-] QUÉ SE GANA, Y POR QUÉ EL CURSO LO EXIGE

Si el bytecode recompilado coincide, el explorador publica la fuente con la marca de verificado y habilita las pestañas para leer y escribir el contrato. Si no coincide, no publica nada. Un contrato sin verificar pide confianza ciega: nadie puede saber qué hace sin descompilarlo. Es el «verificar en vez de creer» del curso aplicado al código propio.

SI EL COMPILADOR NO COINCIDE, FALLA

Es el mismo error de hace dos clases, con otra cara.

HACE DOS CLASES, EN REMIX

«ParserError: Source file requires different compiler version». El archivo fijaba 0.8.34 y el desplegable del navegador tenía otra versión. Se arregló eligiendo la del curso, nunca bajándole el pragma al archivo.

HOY, AL VERIFICAR

Si se declara una versión distinta a la que compiló el bytecode, el explorador recompila, le sale otro bytecode y rechaza la verificación. No dice «versión equivocada»: dice que no coincide, y hay que deducirlo.

De ahí la regla del curso de tener la versión congelada en versiones.md y el pragma fijo en todos los contratos, andamiajes incluidos. Cambiar de compilador no es solo cambiar un número: cambia el objetivo de la EVM, cambia el bytecode y obliga a volver a medir todo.

ABRIR UN CONTRATO YA VERIFICADO

01	ENTRAR	sepolia.etherscan.io/contractsVerified: la lista de los últimos 500 contratos verificados. Abran el primero.
02	PESTAÑA CODE	La fuente completa, la versión del compilador, el optimizador y los argumentos del constructor. Y el ABI, más abajo.
03	PESTAÑA READ CONTRACT	Cada función de lectura, con su botón. Leer no cuesta gas y no pide conectar billetera.
04	PESTAÑA WRITE CONTRACT	Cada función que escribe. Pide conectar billetera y pagar gas: hoy no se toca.

LA PREGUNTA QUE SE HACE MIRANDO LA PANTALLA

¿Qué puede hacer cualquiera con este contrato sin pedirle permiso a nadie, y qué no? Esto es lo que su proyecto tiene que poder mostrar en el Avance 1.

LO QUE EL BLOQUE D DEBIÓ DEJAR

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 ¿Qué parte del despliegue se hizo hoy completa y gratis, y qué parte exacta es la que quedó pendiente?

02 Si corren dos veces el mismo comando de Ignition contra la misma red, ¿qué pasa la segunda vez, y por qué?

03 ¿Qué cuatro cosas hay que entregarle al explorador para verificar, y de qué archivo del proyecto sale cada una?

04 Una verificación falla sin decir por qué. ¿Cuál es la primera sospecha, y qué error de la Sesión 6 se le parece?

LO QUE SALE DE ESTA SESIÓN

EVIDENCIA	QUÉ ES EXACTAMENTE	DÓNDE QUEDA
Defensa	La ficha de lectura llena y la prueba que falló en pantalla.	Planilla del docente, hoy.
Suite	test/s08/Recaudo.test.js con diez pruebas o más, en verde.	Repositorio del equipo.
Mutantes	Los tres muertos, y qué prueba mató a cada uno y por qué.	Captura de cazar-mutantes.js.
Despliegue	La salida de Ignition con la dirección desplegada en local.	Captura de la terminal.

⚠ **LA CAPTURA DEL MUTANTE MUERTO NO BASTA SOLA**

El 35 % del laboratorio está en la explicación, no en la captura: qué prueba mató a cada mutante, y por qué la cobertura no lo veía.

Todo se sube al repositorio del equipo esta semana. Y antes de cada git push: git status y git diff --staged, leyendo cada archivo que va a subir.

EL DESPLIEGUE EN SEPOLIA QUEDA PENDIENTE

Hoy no se hizo. Y sin eso, tres cosas del curso no arrancan.

01	SESIÓN 10	El token ERC-20 se despliega y se verifica en Sepolia. Sin ETH de prueba y sin keystore, ese laboratorio no arranca.
02	SESIÓN 11	El diploma NFT se emite en Sepolia y su tokenURI tiene que resolver desde el explorador.
03	AVANCE 1 · SESIÓN 13	Exige contratos desplegados y verificados. Es nota, y no hay forma de hacerlo la semana anterior.

Esta semana, cada equipo: billetera del curso con ETH de prueba, y el keystore configurado con las tres variables de D.6.

LA PREGUNTA DE LA SEMANA

Si una suite con 88 % de cobertura dejó pasar tres errores, ¿qué número o qué evidencia le pedirían a un proveedor para creerle que su contrato está bien probado?

PARA LA PRÓXIMA SESIÓN

- Terminar el laboratorio 08: suite de diez pruebas o más, cobertura medida, los tres mutantes muertos con su explicación escrita, y la captura del despliegue local con Ignition.
- Configurar el keystore con las tres variables de D.6, y conseguir ETH de prueba en la billetera del curso. Sin esto, la Sesión 10 no se puede hacer.
- Los equipos que defendieron sin la prueba que falla: entregarla con la regla que dijeron en voz alta, antes de la próxima sesión.
- Crear cuenta en ethernaut.openzeppelin.com y resolver el nivel 0: la Sesión 9 empieza en el nivel 1.



LO QUE SE LLEVAN DE HOY

Una prueba verde no demuestra nada si también estaría verde con el error.

Hoy defendieron su contrato, escribieron pruebas y después pusieron esas pruebas a prueba. La próxima sesión cambian de lado: van a atacar contratos ajenos para aprender a defender el propio.

SESIÓN 9 · SEGURIDAD DE CONTRATOS · ETHERNAUT · PARCIAL PRÁCTICO